

გაკვეთილი 4 · 5 საათი

SCSS `@extend`-ი და Placeholder სტილები

რეფაქტორული სტილების კიდევ ერთი გზა — ერთი საბაზისო სტილი რამდენიმე ელემენტზე. დღეს ვისწავლით `@extend`-ის გამოყენებას, `%placeholder`-ების შექმნას და გავარკვევთ, როდის ჯობს `mixin`-ს `extend`.



როგორ ჩაჯდება დღევანდელი **5** საათი

01

გახსენება

variables, nesting და mixin — მოკლე მიმოხილვა წინა მასალიდან

02

ახალი თემა

რა არის @extend და placeholder % — თეორია და მაგალითები

03

შედარება

mixin vs extend — როდის რომელი ინსტრუმენტი ავირჩიოთ

04

ლაივ კოდინგი

info / success / warning box-ები და card-ები რეალურ დროში

05

პრაქტიკა

მარტივიდან საშუალომდე დავალებები — ახალი ცოდნის განმტკიცება

რა არის **@extend**?

მარტივად

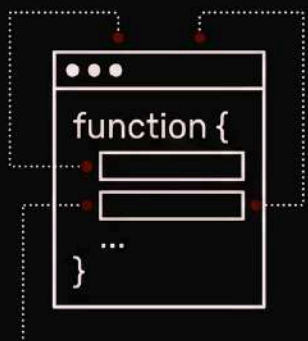
@extend ნიშნავს: აიღე არსებული სტილი და გამოიყენე აქაც. კარგია, როცა რამდენიმე ელემენტს ერთი საერთო ბაზა აქვს — მაგალითად message-box-ებში ან card-ებში.

- ❗ როგორც ერთიანი ფორმა — ყველას ერთი საფუძველი აქვს, მერე კი განსხვავებული დეტალები ემატება.

SCSS მაგალითი

```
.message {  
  color: white;  
  background: #6c5ce7;  
  border: 1px solid #6c5ce7;  
}  
  
.success {  
  @extend .message;  
  // მხოლოდ განსხვავებული ნაწილები  
}
```

mixin vs @extend – რით განსხვავდება?



#FAEBEB

**მიქსინი -
შაბლონი
@include-ით**

საჭიროებს
პარამეტრებს.
ცალკეული CSS.



#FAEBEB

**ექსტენდი -
საერთო
სტილი
კლასიდან**

საერთო
სტილისთვის.
ჯგუფური CSS.

როდის რომელი?

ორივე reusable-ია, მაგრამ სხვადასხვა გზით მუშაობს.

@include mixin

სხვადასხვა ვარიანტები
გჭირდება? პარამეტრები
და მოქნილობა? → mixin
აირჩიე.

@extend

ერთი საერთო base style
გაქვს? ყველა ელემენტს
ერთი ბაზა აქვს? → extend
აირჩიე.

Placeholder – შიდა საბაზისო სტილი

რატომ არის კარგი?

%placeholder არის სტილი, რომელსაც პირდაპირ HTML-ში არ ვიყენებთ — ის მხოლოდ შიდა reusable base style-ია. კოდი სუფთად ორგანიზდება და საბოლოო CSS უფრო გასაგები ხდება.

- იწერება პროცენტის ნიშნით: `%card-base`
- გამოიყენება `@extend`-ით
- HTML-ში არ ჩანს, მაგრამ სტილს აბიარებს

მაგალითი

```
%card-base {  
  background: white;  
  border: 1px solid #ddd;  
  padding: 20px;  
}  
  
.profile-card {  
  @extend %card-base;  
}  
  
.product-card {  
  @extend %card-base;  
}
```

ფერადი **Box**-ები – **info, success, warning**

HTML სტრუქტურა

```
<div class="info-box">
  ინფორმაცია
</div>
<div class="success-box">
  წარმატება
</div>
<div class="warning-box">
  გაფრთხილება
</div>
```

❏ **რჩევა:** ჯერ HTML ააწყვე, მერე placeholder და ბოლოს ფერები.

SCSS კოდი

```
%box-base {
  color: white;
  padding: 16px;
  font-weight: bold;
}

.info-box {
  @extend %box-base;
  background: #0984e3;
}

.success-box {
  @extend %box-base;
  background: #00b894;
}

.warning-box {
  @extend %box-base;
  background: #e17055;
}
```

ერთი base style რამდენიმე box-ზე გადავიდა, მაგრამ ფერები განსხვავებული დარჩა.

პრაქტიკული დავალებები

● მარტივი

შექმენი 3 box: Info, Success, Warning. ყველას საერთო placeholder ჰქონდეს — განსხვავებული იყოს მხოლოდ ფერი.

● საშუალო

შექმენი 2 card: Profile Card და Product Card. ორივეს `%card-base` ჰქონდეს. ტექსტები და ფერები განსხვავებული იყოს.

● დამატებითი

ააწყე პატარა Dashboard: 3 სტატუსის ბლოკი @extend-ით, 2 ღილაკი mixin-ით. ძველი და ახალი ცოდნა ერთად გამოიყენე.

❗ რჩევა: ჯერ იფიქრე — რომელი კოდი მეორდება? სწორედ ის ნაწილი გადაიქცევა placeholder-ად ან mixin-ად.

ხშირი შეცდომები – შეცდომებიდანაც ვსწავლობთ

1

@extend და **@include** ერევათ

თუ mixin-ს იყენებ → `@include`. თუ class ან placeholder-ს აზიარებ → `@extend`. ეს ორი სხვადასხვა ინსტრუმენტია.

2

placeholder შეიქმნა, მაგრამ არ გამოიყენეს

`%card-base` თუ დაწერე, რომელიმე class-მა უნდა დაწეროს `@extend %card-base;` — თორემ უაზრო კოდია.

3

ყველაფერზე **extend**-ს იყენებენ

თუ პარამეტრები გჭირდება, mixin უფრო მოქნილია. ყოველ reusable რამეს თავისი ადგილი აქვს — მთავარია სწორი არჩევანი გააკეთო.



რა უნდა დაიმახსოვრონ დღეს?



Variable

ინახავს ერთ მნიშვნელობას — ფერს, ზომას ან დაშორებას.



Mixin

ინახავს reusable style block-ს და `@include`-ით ვიყენებთ.



Extend

გიზიარებს უკვე არსებულ სტილს class-იდან ან placeholder-იდან.



Placeholder

შიდა base style, რომელიც `%`-ით იწერება და HTML-ში არ ჩანს.

თუ რამდენიმე ელემენტს ერთი და იგივე საბაზისო სტილი აქვს, დიდი შანსია, რომ `@extend` და `placeholder` დაგჭირდება.

კითხვები და სახლის დავალება

კითხვები კლასისთვის

- რას აკეთებს `@extend`?
- რა არის `placeholder`?
- რით განსხვავდება `mixin` და `extend`?
- როდის უნდა გამოვიყენოთ `%card-base`-ის შექმნა?
- რომელი ინსტრუმენტი ინახავს ერთ მნიშვნელობას?

სავარჯიშო დავალება

- შექმენი 3 notification box
- შექმენი 2 card
- გამოიყენე 1 placeholder
- გამოიყენე 1 mixin ღილაკებისთვის
- გამოიყენე მინიმუმ 3 variable

დავალება – Course Cards Section

შექმენი გვერდი, სადაც იქნება 3 სასწავლო ბარათი.

თითო ბარათში უნდა იყოს:

- კურსის სახელი
- მოკლე აღწერა
- ერთი დილაკი

მაგალითად:

- HTML Basics
- SCSS Basics
- Front-end Project

აუცილებლად გამოიყენე

- **variables**
- **nesting**
- **placeholder**
- `@extend`
- **mixin**
- `@include`

მოთხოვნები

1. შექმენი %course-card-base

სამივე ბარათს უნდა ჰქონდეს ერთი საერთო ბაზა placeholder-ით.

2 გამოიყენე @extend

სამივე კურსის ბარათმა უნდა მიიღოს ეს საერთო ბაზა.

3 ერთი ბარათი იყოს გამორჩეული

მაგალითად მესამე ბარათს:

სხვა border სხვა სათაურის ფერი ან thicker border ჰქონდეს

4. ღილაკებისთვის გამოიყენე ერთი mixin

სამივე ღილაკი უნდა იყოს ერთიანი სტილის.

5 შიდა ელემენტები გააფორმე nesting-ით h2 p button

საბოლოოდ რა უნდა გამოვიდეს

3 მსგავსი course card

ერთნაირი სტილი

ერთი გამორჩეული ბარათი

reusable SCSS კოდი

შემაჯამებელი სამუშაო

შექმენი ერთი HTML გვერდი და ერთი SCSS ფაილი.

ნაწილი 1

გვერდზე დაამატე 3 შეტყობინების ბლოკი:

- Info
- Success
- Warning

თითოეულ ბლოკში უნდა იყოს:

- სათაური
- ტექსტი

აუცილებლად გამოიყენე:

- მინიმუმ 3 variable
- 1 placeholder (%box-base)
- @extend
- nesting

ნაწილი 2

გვერდზე დაამატე 2 ბარათი:

- Profile Card
- Product Card

თითოეულ ბარათში უნდა იყოს:

- სათაური
- ტექსტი
- ღილაკი

აუცილებლად გამოიყენე:

- 1 mixin ღილაკისთვის
- @include
- 1 placeholder (%card-base)
- @extend
- variables
- nesting
- &:hover

ერთ-ერთი ბარათი ოდნავ განსხვავებული გააკეთე.

ბონუსი

მოკლედ ახსენი:

- რა არის variable
- რა არის mixin
- რა არის @extend
- რა არის %placeholder

